ELEC 475

Lab 3 - Pet Nose Localization Correction

Connor Leung 20322809

Ryan Su 20344856

1. Model

1.1 CorrectionNet

CorrectionNet is designed with four convolution blocks followed by two fully connected layers. The final layer has no activation because offset can be negative.

The output (t_u, t_v) is normalized offset from the patch centre to the nose. The corrected prediction is stated in the equation:

$$(\hat{u}', \hat{v}') = (c_u, c_v) + 64(t_u, t_v)$$

Where (c_u, c_v) is the first prediction rounded to the nearest pixel and 64 is half the 128x128 patch size.

Layer	Definition	Output shape	Params
Input	-	(3, 64, 64)	0
Block 1	Conv2d(3, 32, 3, p=1) → BN(32) → ReLU → MaxPool(2, 2)	(32, 32, 32)	960
Block 2	Conv2d(32, 64, 3, p=1) → BN(64) → ReLU → MaxPool(2, 2)	(64, 16, 16)	18,624
Block 3	Conv2d(64, 128, 3, p=1) → BN(128) → ReLU → MaxPool(2, 2)	(128, 8, 8)	74,112
Block 4	Conv2d(128, 128, 3, p=1) → BN(128) → ReLU → MaxPool(2, 2)	(128, 4, 4)	147,840
Flatten	torch.flatten(x, 1)	2,048	0
fc1	Linear(2048, 256) → ReLU	256	524,544
fc2	Linear(256, 2)	2	514
Total			766,594

Table 1: CorrectionNet architecture (766,594 parameters).

1.2 CorrectionNetModified

CorrectionNetModified has the same input, output, number of blocks and pooling schedule which keeps a 4x4 grid before flattening. The modifications are:

1. Two convolutions per block (VGG-style) where each block applies Conv → BN → ReLU twice before pooling which doubles the length to eight convolution layers and increases the receptive field at each resolution
2. Block 4 now has 256 output channels from 128 which doubles the flattened feature vector from 2048 to 4096.
3. FC1 is now followed by BatchNorm1d(256), then ReLU and Dropout(p = 0.3) before FC2. BatchNorm1d is added to keep the larger head from collapsing to a constant zero offset. Originally, most FC1 units were negative for every input

preventing the gradient from changing. Re-centring them on every batch resolved this.

Global average pooling was not used because the position of the nose within the 4x4 grid is where the model needs to regress to which allows the spatial layout to be kept after flattening.

Layer	Definition	Output shape	Params
Input	—	(3, 64, 64)	0
Block 1	[Conv2d(3, 32) → BN → ReLU] → [Conv2d(32, 32) → BN → ReLU] → MaxPool(2, 2)	(32, 32, 32)	10,272
Block 2	[Conv2d(32, 64) → BN → ReLU] → [Conv2d(64, 64) → BN → ReLU] → MaxPool(2, 2)	(64, 16, 16)	55,680
Block 3	[Conv2d(64, 128) → BN → ReLU] → [Conv2d(128, 128) → BN → ReLU] → MaxPool(2, 2)	(128, 8, 8)	221,952
Block 4	[Conv2d(128, 256) → BN → ReLU] → [Conv2d(256, 256) → BN → ReLU] → MaxPool(2, 2)	(256, 4, 4)	886,272
Flatten	torch.flatten(x, 1)	4,096	0
fc1	Linear(4096, 256) → BatchNorm1d(256) → ReLU → Dropout(0.3)	256	1,049,344
fc2	Linear(256, 2)	2	514
Total			2,224,034

Table 2: CorrectionNetModified architecture (2,224,034 parameters). All Conv2d layers are 3 × 3 with padding 1.

2. Training

Four correction models were trained using train_correction.py and each model (CorrectionNet and CorrectionNetModified) was trained twice with otherwise identical settings: once with augmentation and once with the --no-augment flag. The --modified flag selects CorrectionNetModified. Since a training image has no first prediction, one is simulated by placing the patch centre at a random offset from the true nose.

Setting	Value
Training data	5,494 labelled training images (train_noses.txt). The test set is never used.
Samples per epoch	One random patch per training image per epoch (5,494 patches); new random centres every epoch
Patch centre	(c_u, c_v) = round(u + U(−40, 40), v + U(−40, 40)), drawn independently for u and v
Patch extraction	128 × 128 crop of the original RGB image centred on (c_u, c_v) with img.crop() (area outside the image is black), resized to 64 × 64 with Image.BILINEAR
Input normalization	ToTensor, then ImageNet mean [0.485, 0.456, 0.406] and std [0.229, 0.224, 0.225]
Target	$\frac{(u-c_u)}{64}, \frac{(v-c_v)}{64}$ roughly in [−0.63, 0.63]
Flip (aug only)	Horizontal flip with p = 0.5; the target becomes $(-t_u, t_v)$
Colour jitter (aug only)	ColorJitter(brightness=0.3, contrast=0.3, saturation=0.3); target unchanged
Loss	MSE between predicted and target normalized offsets
Optimizer	Adam, lr = 1e−3, weight decay = 1e−5 (default betas (0.9, 0.999))
LR schedule	CosineAnnealingLR, $T_{max} = 20$, stepped once per epoch (lr decays from 1e−3 towards 0)
Epochs / batch size	20 epochs, batch size 64, shuffled each epoch
Reproducibility	Seed 42 for Python random, NumPy and PyTorch; DataLoader with 4 workers, per-worker seeding (seed_worker) and a generator seeded with 42
Hardware	Apple M4 Pro GPU through PyTorch's MPS backend

Table 3: Correction model training details.

The four models were trained with the following commands (also in commands.txt):

```
python train_correction.py --no-augment --save-model checkpoints/correction_noaug.pth --plot-file plots/loss_correction_noaug.png
python train_correction.py --save-model checkpoints/correction_aug.pth --plot-file plots/loss_correction_aug.png
python train_correction.py --modified --no-augment --save-model checkpoints/correction_mod_noaug.pth --plot-file plots/loss_correction_mod_noaug.png
python train_correction.py --modified --save-model checkpoints/correction_mod_aug.pth --plot-file plots/loss_correction_mod_aug.png
```

At test time, correct.py extracts one patch centred on each first prediction (rounded to the nearest pixel), runs the correction model once in eval mode, and applies the equation in Section 1. No test-set result was used to choose any setting.

3. Outputs

3.1 Accuracy and cost tables

Errors are Euclidean distances in original-image pixels over the 698 test images.

Model	Min	Mean	Median	Max	Std
Baseline (mean)	0.6	79.7	70.0	848.2	56.1
SnoutNet	0.8	57.0	44.3	1084.5	57.4
SnoutNet-A	1.8	44.8	37.0	690.0	39.6
SnoutNet-V	0.4	12.9	8.7	175.5	17.3
Ensemble (mean)	0.7	31.2	23.5	597.2	33.7
Ensemble (median)	0.4	29.4	20.2	676.3	38.0
SnoutNet + corr.	0.2	43.8	21.0	1066.9	61.2
SnoutNet + corr. (aug)	0.3	43.6	19.9	1069.2	61.3
SnoutNet + mod. corr.	0.3	40.7	13.9	1071.3	62.7
SnoutNet + mod. corr. (aug)	0.2	40.3	14.4	1071.5	62.3
SnoutNet-V + corr.	0.4	12.3	7.8	201.0	17.4
SnoutNet-V + corr. (aug)	0.3	12.2	7.7	200.1	17.0
SnoutNet-V + mod. corr.	0.0	9.6	5.2	206.0	17.1
SnoutNet-V + mod. corr. (aug)	0.2	9.7	5.2	201.4	17.4
Ensemble (mean) + corr.	0.3	20.9	10.7	597.7	34.6
Ensemble (mean) + corr. (aug)	0.1	20.7	10.2	599.0	34.9
Ensemble (mean) + mod. corr.	0.3	17.0	6.5	598.9	34.8
Ensemble (mean) + mod. corr. (aug)	0.2	17.0	6.5	597.0	34.7

Table 4: Accuracy (pixels). "corr." = CorrectionNet, "mod. corr." = CorrectionNetModified, "(aug)" = trained with augmentation.

Model	Params	ms/img
Baseline (mean)	0	0.0
SnoutNet	5,617,794	1.3

Model	Params	ms/img
SnoutNet-A	57,012,034	1.7
SnoutNet-V	134,268,738	8.3
Ensemble (mean)	196,898,566	11.4
Ensemble (median)	196,898,566	11.4
SnoutNet + corr.	6,384,388	2.2
SnoutNet + corr. (aug)	6,384,388	2.0
SnoutNet + mod. corr.	7,841,828	2.3
SnoutNet + mod. corr. (aug)	7,841,828	2.3
SnoutNet-V + corr.	135,035,332	8.9
SnoutNet-V + corr. (aug)	135,035,332	9.0
SnoutNet-V + mod. corr.	136,492,772	9.4
SnoutNet-V + mod. corr. (aug)	136,492,772	9.3
Ensemble (mean) + corr.	197,665,160	12.0
Ensemble (mean) + corr. (aug)	197,665,160	12.0
Ensemble (mean) + mod. corr.	199,122,600	12.4
Ensemble (mean) + mod. corr. (aug)	199,122,600	12.3

Table 5: Cost. Corrected models include the first model(s) plus the correction model.

3.2 Cumulative error distribution

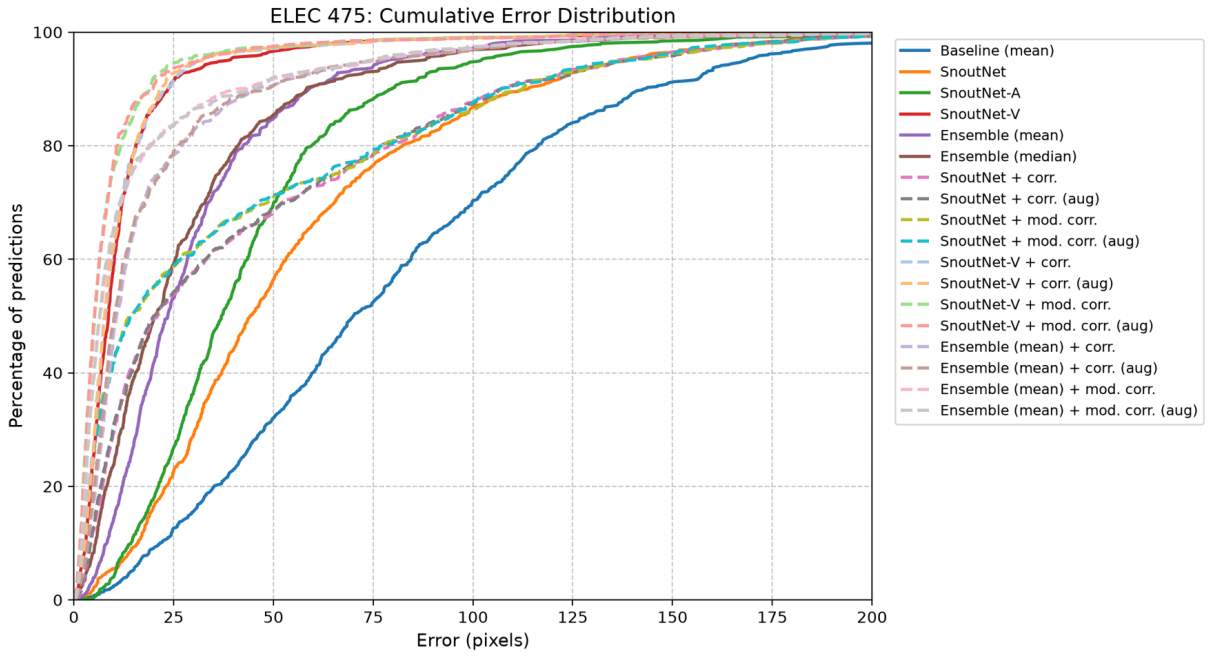


Figure 1: Cumulative error distribution (plots/ced.png). Solid = In-Lab models, dashed = corrected.

3.3 Best and worst predictions



Figure 2: SnoutNet (best_worst_snoutnet.png). Green circle = ground truth, red cross = prediction.



Figure 3: SnoutNet + CorrectionNet, no augmentation (best_worst_snoutnet_correction_noaug.png).



Figure 4: SnoutNet + CorrectionNet with augmentation (best_worst_snoutnet_correction_aug.png).

3.4 Loss curves

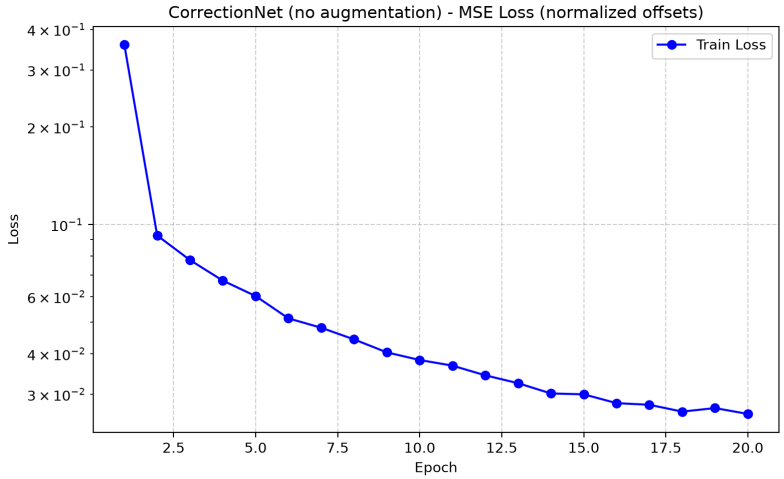


Figure 5: Training loss, CorrectionNet, no augmentation (loss_correction_noaug.png).

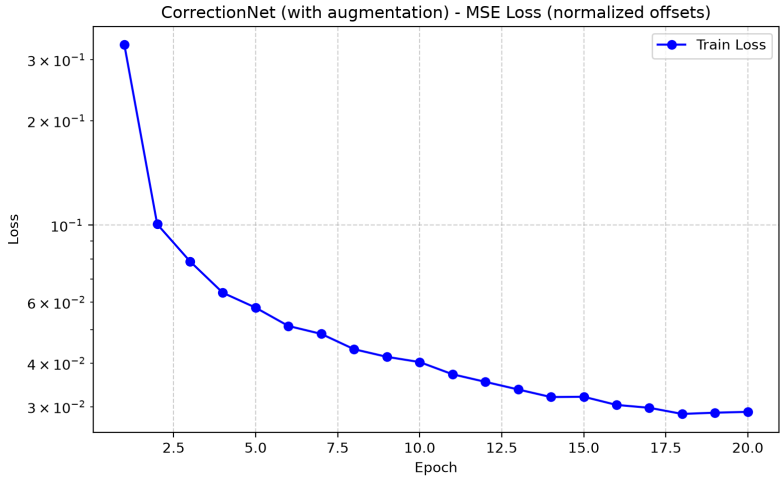


Figure 6: Training loss, CorrectionNet, with augmentation (loss_correction_aug.png).

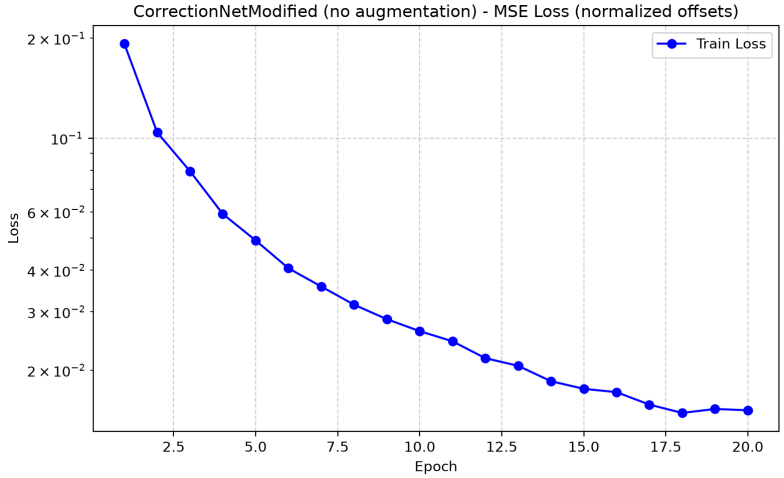


Figure 7: Training loss, CorrectionNetModified, no augmentation (loss_correction_mod_noaug.png).

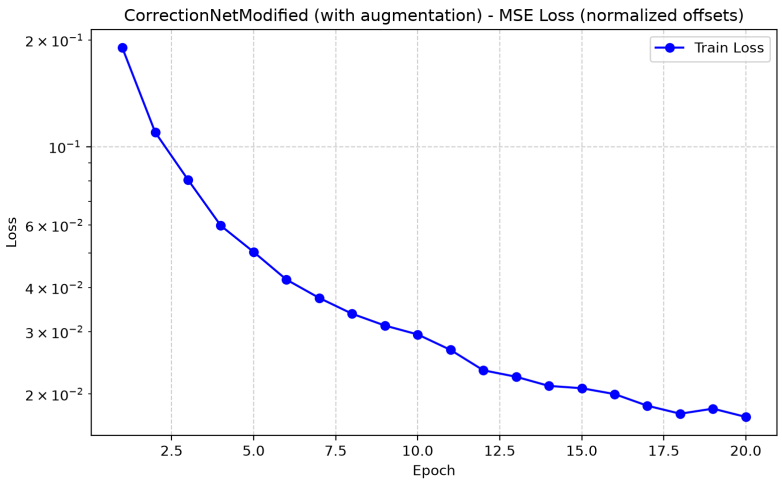


Figure 8: Training loss, CorrectionNetModified, with augmentation (loss_correction_mod_aug.png).

4. Analysis

4.1 Failure cases

All models share similar failure points, primarily in certain test data where the animals are in unusual poses where the head is not facing the camera. The images that cause the most failures are a cat lying on its side (Egyptian_Mau_165, the worst image for both SnoutNet and SnoutNet-A, at 1,084 px and 690 px), a cat walking with its head down near the floor (Egyptian_Mau_146), a puppy sniffing the ground (american_bulldog_217), and a dog curled up asleep (wheaten_terrier_153). Others have low contrast around the nose, such as a black Newfoundland whose dark nose sits in dark fur (newfoundland_116), or a small animal far from the camera with a person in the scene (scottish_terrier_109, which appears in the worst four of both SnoutNet-A and SnoutNet-V).

The failure is to be expected because most training images show a roughly front-facing pet with its head near the centre. The models can then reach a low training loss by learning that “the nose is in the middle of the face which is also near the middle of the image”. When the head is at the edge of the frame, the body’s features dominate. This is a flaw with SnoutNet where it predicts a particular point on the body (usually in the centre of the image). Animals with distinct patterns such as the patterned coats of the Egyptian Mau, include dark blob features which can be mistaken for a nose. Lastly, the images are resized to 227x227 with different horizontal and vertical factors meaning that any details of noses are then shrunk to a few pixels around the nose.

4.2 Pretraining

SnoutNet-V has a mean error of 12.9 px, compared to the SnoutNet model trained from scratch with a 57.0 px mean error. SnoutNet-V’s worst error was 176 px compared to the one from scratch with a 1,085 px. Since ImageNet contains over a hundred dog breeds and several cat classes, the pretrained features already respond to animal

faces, eyes, and noses, and factor in pose, lighting, and background. By fine-tuning from ImageNet weights, the model needed to learn how to map those features to a position. SnoutNet must learn all of this from 5,494 images, which misses out on learning general-purpose features, causing it to rely on simpler features such as image centre. SnoutNet-A is also pretrained but improves much less (44.8 px), which hints that depth matters as well. VGG16 keeps more spatial resolution through 3x3 convolutions; whereas AlexNet's large early strides discard the fine positional information needed for localization.

4.3 Ensembles

The use of ensembles did not improve the models. The mean ensemble (31.2 px) and the median ensemble (29.4 px) are both much worse than SnoutNet-V alone (12.9 px), but both beat SnoutNet and SnoutNet-A. An ensemble only outperforms its best member when the members have similar accuracy, and their errors are not strongly correlated which allows the average to cancel independent errors. In the SnoutNet-V case, it is four times more accurate than the other two, therefore the averaging mostly pulls its good predictions towards the worse cases. The errors of SnoutNet and SnoutNet-A are fairly correlated (correlation coefficient of 0.73 between their per-image errors), so they both struggle on the same images and averaging does not cancel each other out. The median ensemble is slightly better than the mean because, with three members, the median ignores the most extreme of the three predictions in each coordinate. The expected result usually is that the ensemble beats the best member if combined with several models of similar quality ie: VGG16 models using different seeds or augmentations.

4.4 Correction model

The correction model helps fill the gap once the model is already close to the nose and does not provide any benefit when the nose is far away. Throughout training, the patch centre is at most 40 px from the nose in each coordinate, and the 128x128 patch only covers +-64 px. Therefore, if the first prediction is more than about 64 px away, the nose is not in the patch.

First model	First error	Images	Before	corr. (aug)	mod. corr. (aug)
SnoutNet	< 10 px	38	5.6	7.3	5.1
SnoutNet	10-40 px	270	26.3	10.5	6.9
SnoutNet	40-64 px	173	50.6	27.5	21.9
SnoutNet	≥ 64 px	217	109.4	104.0	102.6
SnoutNet-V	< 10 px	411	5.5	7.5	5.3

First model	First error	Images	Before	corr. (aug)	mod. corr. (aug)
SnoutNet-V	10-40 px	256	16.8	12.4	9.4
SnoutNet-V	≥ 40 px	31	77.7	72.0	70.4
Ensemble (mean)	< 10 px	100	6.6	7.1	5.7
Ensemble (mean)	10-40 px	441	22.8	11.0	7.4
Ensemble (mean)	40-64 px	98	50.1	29.2	23.1
Ensemble (mean)	≥ 64 px	59	104.2	101.8	97.6

Table 6: Mean error (px) before and after correction, grouped by the error of the first prediction. SnoutNet-V has only 31 images above 40 px, so those groups are combined.

Comparing the three first models, the correction helps most when the first prediction lands 10-64 px from the nose, i.e. inside the patch but not on the nose. The mean ensemble gains the most: 441 of its 698 predictions fall in the 10-40 px range, and its mean error drops from 31.2 px to 20.7 px with CorrectionNet (aug) and to 17.0 px with CorrectionNetModified (aug), while its median falls from 23.5 px to 6.5 px. SnoutNet's median also improves strongly (44.3 → 14.4 px), but its mean only drops from 57.0 px to 40.3 px because 217 of its predictions are 64 px or more away, where the nose is outside the patch and the error barely changes (109.4 → 102.6 px). SnoutNet-V gains the least in absolute terms (12.9 → 9.7 px) because 411 of its predictions are already within 10 px. For these near-perfect predictions, the original CorrectionNet makes things slightly worse (5.5 → 7.5 px for SnoutNet-V): training patches are offset by up to 40 px, so the model rarely sees a nose already at the centre, and its few pixels of regression noise exceed the remaining error. CorrectionNetModified avoids this (5.5 → 5.3 px). The maximum errors are essentially unchanged for all three first models, since those predictions are far outside the patch.

4.5 Augmentation

Augmentation had little effect on the result. With CorrectionNet, the mean error changed at most by 0.2 px (SnoutNet 43.8 → 43.6, SnoutNet-V 12.3 → 12.2, ensemble 20.9 → 20.7) and using CorrectionNetModified, the differences were within ±0.4 px. The loss curves look similar to the non-augmented runs with the augmented runs having slightly higher training loss (0.029 vs. 0.027 for CorrectionNet and 0.017 vs. 0.016 for CorrectionNetModified) which is expected for a harder training task.

The small change with augmentation may be a result that the random patch centre is already augmented strongly and with every epoch, each image produces a new patch with a new offset. Therefore, the model rarely sees the same input twice and does not overfit much in 20 epochs. With horizontal flipping, the symmetry is already trained because pets face both left and right and a nose is similar when mirrored. The colour

jitter mainly helps with lighting changes, but the test images come from the same distribution as training images, so the robustness to colour shifts has little to improve on. Augmentation would likely improve more with longer training or smaller training set.

4.6 Modified correction model

The modifications doubled the convolutions in each block (VGG-style), widened block 4 to 256 channels, and added BatchNorm1d and dropout to the FC head. The resulting correction model changed from 0.77M parameters to 2.22M parameters but only added 0.1-0.5 ms per image. The modification improved performance for every first model and with augmentation, mean error went from 43.6 to 40.3 px for SnoutNet, 12.2 to 9.7 px for SnoutNet-V, and 20.7 to 17.0 px for the mean ensemble. The medians improved significantly: SnoutNet-V 7.7 → 5.2 px, ensemble 10.2 → 6.5 px. The most important change is that the modified model does not make already-good predictions worse and its final training loss is about 40% lower (0.017 vs. 0.029, both with augmentation). This is because the two stacked 3x3 convolutions give a larger receptive field and more non-linearity at each resolution which allows the network to recognize the nose from its context before the spatial resolution is reduced. The extra channels and FC1 inputs keep more precise position information. Also, BatchNorm1d stabilizes training of the larger head and dropout limited overfitting. Like the original model, it cannot fix predictions that are outside the patch causing the maximum errors to be unchanged.